CYBERBULLYING DETECTION IN YOUTUBE COMMENTS

Submitted by,

Rifa Fathima

Roll no: 18

MSc Data Science

ABSTRACT

This project focuses on detecting cyberbullying in YouTube comments using machine learning techniques. A dataset of manually collected comments were cleaned, preprocessed, and labeled into two categories: *cyberbullying* and *not cyberbullying*. Text preprocessing steps such as cleaning, stopword removal, tokenization, emoji-to-text conversion, and lemmatization were applied. The SVM model achieved the highest accuracy of approximately **83%**. Results indicate that classical ML approaches combined with proper preprocessing can effectively identify harmful content in social media platforms.

INTRODUCTION

Background

Cyberbullying refers to the use of digital communication platforms to harass, insult, or threaten individuals. With the rapid growth of social media, cyberbullying has become a serious concern impacting mental health, safety, and digital well-being. YouTube comment sections often contain toxic behaviour, making them suitable for cyberbullying research.

Importance of Cyberbullying Detection

- Protects users from online harassment
- Helps platforms maintain safer digital environments
- Supports automated moderation systems

Why YouTube Comments?

- YouTube has millions of daily active users and large volume of user-generated content
- High presence of insults, mocking, harsh criticism and abusive language
- Real-world dataset suitable for classification tasks

PROBLEM DEFINITION

To classify YouTube comments as either:

- **Cyberbullying (1)**
- **Non-cyberbullying (0)**

using machine learning.

Scope of the Study

- Binary classification
- English language comments
- Emoji-aware preprocessing
- Classical ML models

Objectives

- To Collect YouTube comments
- To Preprocess and clean the dataset
- Extract meaningful text features
- To Train and compare classifiers
- To Evaluate performance

Limitations

- Limited dataset size
- Sarcasm and subtle bullying may be misclassified
- Only English considered
- Imbalanced labels may affect performance
- No deep learning models used due to resource constraints

SYSTEM REQUIREMENTS / TOOLS USED

Programming Language

- Python 3

Libraries & Tools

- Pandas
- NumPy
- NLTK
- Emoji
- Scikit-learn
- Matplotlib
- Seaborn
- Joblib

Platform

- Google Colab (Jupyter Notebook Environment)

Dataset Source

- Manually scraped YouTube comments (after collection stored as csv)

DATASET DESCRIPTION

Source of Comments

YouTube comments scraped from selected videos with active comment sections.

Dataset Size

- Total Comments: **[1000]**
- Cyberbullying: **[500]**
- Not cyberbullying: **[500]**

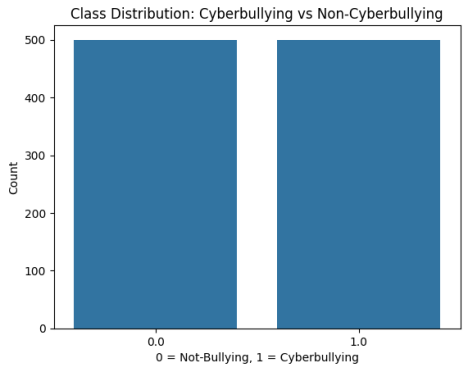
Label Description

- 1 = Cyberbullying
- 0 = Not cyberbullying

Examples of Comments

Comment	Label
“Bro you look so stupid 🤡”	Cyberbullying
“This is such an amazing tutorial”	Not cyberbullying
“Your voice is irritating 😂😭”	Cyberbullying

Data Distribution Plot



Preprocessing Requirements

- Emoji interpretation
- Removal of unwanted characters
- Handling stopwords
- Normalization

METHODOLOGY

1. Data Preprocessing

Steps applied:

1. Convert to lowercase
2. Remove URLs
3. Remove @mentions & hashtags
4. Convert emojis to meaningful text using emoji.demojize()
5. Remove punctuation & special characters
6. Tokenize using NLTK
7. Remove stopwords
8. Lemmatize words
9. Rejoin into final cleaned text

```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    # Lowercase
    text = text.lower()

    # Remove URLs
    text = re.sub(r'http\S+|www\S+', '', text)

    # Remove @mentions or #hashtags
    text = re.sub(r'@\w+|#\w+', '', text)

    # Convert emojis to text meaning
    text = emoji.demojize(text, delimiters=(" ", " "))

    # Remove special characters except emoji words
    text = re.sub(r'[^a-zA-Z0-9_ ]', ' ', text)

    # Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()

    # Tokenization + stopword removal + lemmatization
    words = [
        lemmatizer.lemmatize(w)
        for w in text.split()
        if w not in stop_words
    ]

    return " ".join(words)
```

2. Feature Extraction — TF-IDF

TF-IDF Vectorizer:

- max_features = 5000
- ngram_range = (1, 2)

It converts the cleaned text into numerical form for ML algorithms.

```
from sklearn.feature_extraction.text import TfidfVectorizer

X = df["cleaned_text"]
y = df["Label"]

tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2))
X_tfidf = tfidf.fit_transform(X)

print("TF-IDF shape:", X_tfidf.shape)
```

```
TF-IDF shape: (1000, 5000)
```

3. Machine Learning Models

The following models were trained and compared:

- Logistic Regression
- Support Vector Machine (LinearSVC)
- Naive Bayes
- Random Forest

Train-test split:

- 80% training
- 20% testing
- Stratified to maintain class balance

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Data Ready for Modeling!")
```

```
Data Ready for Modeling!
```

```python
from sklearn.svm import LinearSVC

model_svm = LinearSVC()
model_svm.fit(X_train, y_train)

y_pred_svm = model_svm.predict(X_test)

print("SVM RESULTS")
print(classification_report(y_test, y_pred_svm))
```

IMPLEMENTATION

1. Importing Libraries

Imports all required Python libraries for NLP, ML, and visualization.

- **Pandas & NumPy** → for handling tabular data
- **NLTK** → for tokenization, stopword removal, and lemmatization
- **Emoji** → for converting emojis into descriptive text
- **Scikit-learn** → for TF-IDF vectorization, ML models, evaluation
- **Matplotlib & Seaborn** → for plotting graphs and confusion matrices
- **Joblib** → for saving and loading models

2. Dataset Loading

Reads the CSV file and removes null/duplicate rows.

3. Preprocessing Function

A custom cleaned_text() function was built to standardize and clean raw user comments. It includes:

- **Lowercasing**
  Converts all characters to lowercase for uniformity.

- **Removing URLs, @mentions, and hashtags**
  Removes noise commonly found in online comments.

- **Emoji Handling**
  Uses emoji.demojize() to convert emojis into text
  Example: 😂 → "face_with_tears_of_joy"
  This is important because emojis often indicate mockery or bullying tone.

- **Removing Special Characters**
  Keeps only alphabets, numbers, and emoji words.

- **Stopword Removal**
  Removes common but meaningless words like "the", "is", "are", etc.

- **Tokenization & Lemmatization**
  Breaks text into words and reduces them to their dictionary root forms.

- **Final Cleaned Text**
  Tokens are joined back into a clean sentence that is ready for modeling.

This module forms the most important part of the preprocessing pipeline.

4. TF-IDF Vectorization

Converts cleaned textual data into numerical feature vectors.

```
from sklearn.feature_extraction.text import TfidfVectorizer

X = df["cleaned_text"]
y = df["Label"]

tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2))
X_tfidf = tfidf.fit_transform(X)

print("TF-IDF shape:", X_tfidf.shape)
```

```
TF-IDF shape: (1000, 5000)
```

TF-IDF was chosen because:

- It captures important words
- It reduces the impact of rare/noisy words
- It works very well with models like SVM and Logistic Regression

The output is a sparse feature matrix suitable for machine learning algorithms.

5. Train-Test Split

The dataset is split using stratified sampling:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Data Ready for Modeling!")
```

```
Data Ready for Modeling!
```

- 80% data → Training
- 20% data → Testing

- Stratify = y ensures equal distribution of labels in both sets.

This prevents model bias toward the majority class.

6. Model Training

Four ML models were trained:

1. **Logistic Regression**
2. **SVM (LinearSVC)**
3. **Naive Bayes**
4. **Random Forest**

Each model was fitted on the TF-IDF vectors:

The SVM model achieved the highest accuracy (83%).

7. Model Evaluation

Evaluation metrics used:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

The classification_report() function provided a detailed performance analysis.

Confusion matrices were plotted using Seaborn heatmaps for easy visualization.

8. Saving the Best Model (SVM)

Uses Joblib to save:

- SVM model
- TF-IDF vectorizer

```
import joblib

# Save the SVM model
joblib.dump(model_svm, "svm_model.pkl")

# Save the TF-IDF vectorizer also
joblib.dump(tfidf, "tfidf_vectorizer.pkl")

print("Model and vectorizer saved successfully!")
```

```
Model and vectorizer saved successfully!
```

These files allow the model to be reused without retraining.

9. User Input Prediction

This module loads the saved model and vectorizer, then allows the user to enter any comment:

```
model_svm = joblib.load("svm_model.pkl")
tfidf = joblib.load("tfidf_vectorizer.pkl")

print("Model and vectorizer loaded!")
```

```
Model and vectorizer loaded!
```

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'@\w+|#\w+', '', text)
    text = emoji.demojize(text, delimiters=(" ", " '
    text = re.sub(r'[^a-zA-Z0-9_ ]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()

    words = [
        lemmatizer.lemmatize(w)
        for w in text.split()
        if w not in stop_words
    ]
    return " ".join(words)
```

```python
def predict_comment(text):
    cleaned = clean_text(text)
    vectorized = tfidf.transform([cleaned])
    prediction = model_svm.predict(vectorized)[0]

    if prediction == 1:
        return "⚠️ Cyberbullying comment"
    else:
        return "✔️ Not cyberbullying comment"
```

```python
user_text = input("Enter a comment to classify: ")

result = predict_comment(user_text)
print("\nPrediction:", result)
```

```
Enter a comment to classify: You are so dumb🤣🤣🤣

Prediction: ⚠️ Cyberbullying comment
```

The comment is cleaned using the same preprocessing steps, transformed using TF-IDF, and passed into the model for prediction:

- Output **Cyberbullying (1)**
- Output **Not cyberbullying (0)**

This simulates how the model would work in a real-time application.

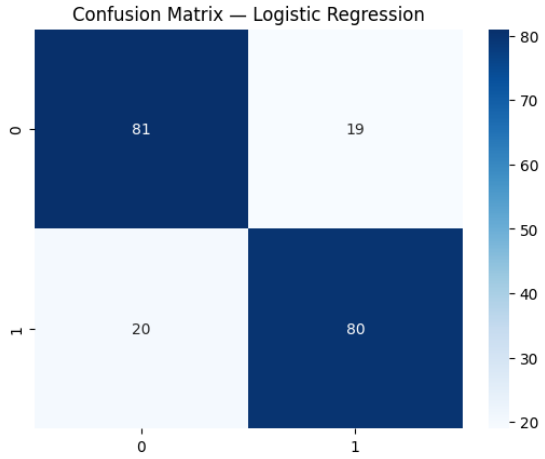
RESULTS AND DISCUSSION

Evaluation Metrics

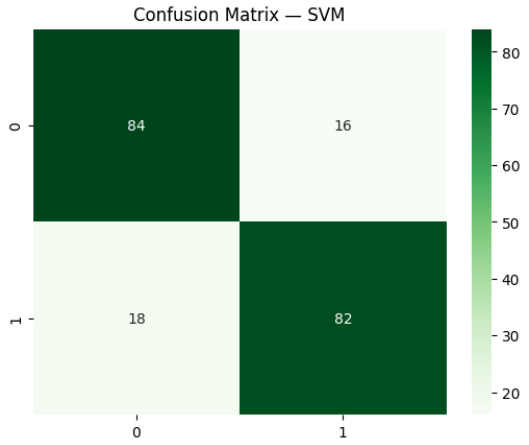
Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	80%	80%	80%	80%
SVM (Best Model)	**83%**	83%	82%	82%
Naive Bayes	79%	86%	69%	76%
Random Forest	79%	73%	90%	81%

Confusion Matrix

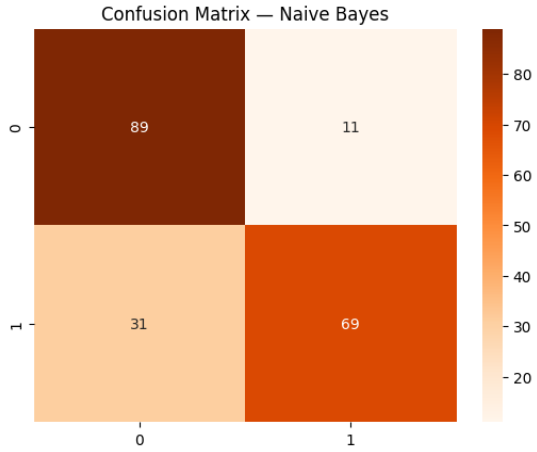
- **Logistic Regression**



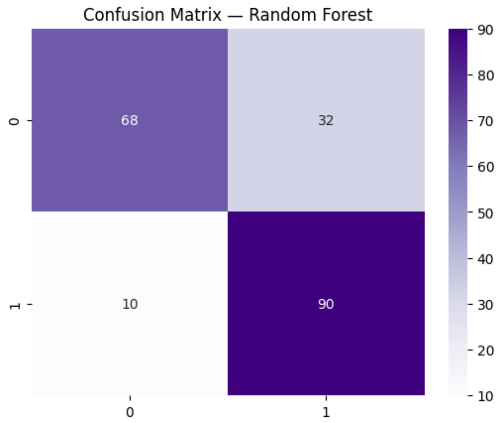
- **SVM**



- **Naïve Bayes**



- **Random Forest**



Discussion

- SVM performs best because TF-IDF features align well with linear decision boundaries.
- Naive Bayes works well for text but suffers due to emojis/slang.
- Random Forest underperforms due to sparse high-dimensional data.
- Proper preprocessing significantly boosts accuracy.

FUTURE ENHANCEMENTS

- Add deep learning models (LSTM, BERT)
- Real-time comment monitoring system
- Multilingual bullying detection
- Severity-level classification
- More diverse and larger dataset

CONCLUSION

This project successfully implemented a cyberbullying detection system for YouTube comments using machine learning. With TF-IDF features and classical algorithms, the SVM model achieved the highest accuracy (~83%). The system can be extended and improved with larger datasets and deep learning models.

REFERENCES

- Scikit-learn Documentation
- NLTK Documentation
- Emoji Python Library
- YouTube comment scraping API